



PROGETTO BASE DI DATI

Martina Lumastro

AeroBase - Sistema ERP per la Gestione Aeroportuale Multi-Tenant



Indice

1. Descrizione del contesto e funzionalità	3
2. Descrizione in linguaggio naturale.....	3
3. Obiettivi del progetto.....	4
4. Utenti del sistema	4
5. Analisi concettuale generale	5
6. Generalizzazione e Specializzazione	6
6.1 Generalizzazione Utente	6
6.2 Specializzazione Volo.....	6
7. Diagramma E-R.....	7
8. Entità e attributi	7
8.1 Tabelle Classiche	7
8.2 Tabelle di Specializzazione Volo.....	9
8.3 Relazioni con attributi	10
9. Progettazione logica relazionale	10
10. Vincoli del sistema ed implementazione nel DB MySQL.....	11
10.1 Vincoli di Integrità e Unicità	11
10.2 Vincoli di Dominio e Vincoli CHECK.....	12
10.3 Vincoli sulle Specializzazioni e Trigger SQL	12
10.4 Politiche di Cancellazione (ON DELETE)	14
10.5 Indici e Ottimizzazione	14
11. Vincoli applicativi gestiti da Django	14
12. Gestione ruoli e isolamento multi-tenant.....	15
13. Implementazione del sistema informativo	15
14. Workflow del sistema	17
15. Motivazioni progettuali e architetture	18
16. Conclusione	18

1. Descrizione del contesto e funzionalità

Il progetto riguarda la progettazione e lo sviluppo di un sistema informativo per la gestione integrata di servizi aeroportuali. AeroBase si configura come una piattaforma web e gestionale di tipo ERP, strutturata per operare in modalità **multi-tenant**: un'unica infrastruttura centrale serve molteplici scali aeroportuali, garantendo il completo isolamento dei dati operativi tra le diverse strutture. Ciascun aeroporto può quindi amministrare in autonomia i propri voli, i gate e il personale dipendente.

Il sistema unisce due anime funzionali:

1. **Un portale pubblico**, dedicato ai passeggeri, per la consultazione dei voli in tempo reale, la prenotazione dei biglietti, la scelta dei posti e la gestione dei bagagli e dei pagamenti.
2. **Un'area gestionale riservata**, dedicata agli operatori interni, per la supervisione logistica, il controllo dei voli, le procedure di check-in e l'amministrazione del personale.

L'intero sistema poggia su un database relazionale MySQL per la persistenza ed è accessibile tramite un'applicazione web sviluppata con il framework Django (Python) per la logica di backend, unita a un'interfaccia dinamica in HTML, CSS e JavaScript.

2. Descrizione in linguaggio naturale

AeroBase offre un ventaglio diversificato di funzionalità a seconda della tipologia di utente:

- **I passeggeri** possono consultare le tratte disponibili, effettuare ricerche mirate (per aeroporto di partenza, destinazione e data) e completare il flusso di prenotazione. In questa fase selezionano la classe di viaggio, riservano un posto specifico, dichiarano i bagagli ed eseguono una transazione finanziaria simulata, con la facoltà di salvare in sicurezza il metodo di pagamento utilizzato.
- **Il personale aeroportuale** accede alle sole funzionalità di competenza tramite un sistema di controllo degli accessi basato sui ruoli (*Role-Based Access Control*):
 - **L'operatore voli** gestisce la logica dei voli del proprio scalo di appartenenza, modificando orari, assegnazioni dei gate, indicando eventuali ritardi e aggiornando lo stato operativo.
 - **L'operatore bagagli** ricerca le prenotazioni dei passeggeri durante il check-in, effettua le verifiche fisiche sul peso e sulla tipologia di bagaglio e ne aggiorna lo stato logistico.
 - **L'amministratore (Admin) aeroportuale** coordina il personale del proprio scalo, abilitando o disabilitando gli account degli operatori e assegnando loro i relativi ruoli.

- **Gli utenti non autenticati** possono consultare liberamente la home page, cercare voli e visualizzare il *tabellone voli*. Quest'ultimo riproduce le partenze e gli arrivi giornalieri e viene aggiornato in tempo reale simulato tramite richieste asincrone JavaScript verso endpoint Django.

A livello informativo, il sistema modella in modo approfondito la distinzione logica tra **voli nazionali** (che possono includere agevolazioni statali legate al codice fiscale del passeggero) e **voli internazionali** (che richiedono il controllo del passaporto, l'indicazione di visti, fusi orari ed eventuali certificazioni sanitarie).

3. Obiettivi del progetto

- Analizzare il dominio aeroportuale identificando entità, attributi, vincoli e relazioni.
- Progettare un modello concettuale coerente espresso tramite un diagramma Entità-Relazione (E-R).
- Derivare il modello logico relazionale, normalizzando le tabelle e ottimizzando le associazioni.
- Implementare fisicamente il database in MySQL.
- Sviluppare l'applicazione web con Django, integrandola stabilmente con la base dati MySQL.
- Configurare una robusta profilazione degli utenti basata sui ruoli e implementare la logica multi-tenant.
- Implementare il flusso completo lato passeggero (ricerca, prenotazione, posto, bagaglio, transazione).
- Creare i flussi operativi di audit log, controllo bagagli e monitoraggio dei voli.
- Sviluppare un tabellone voli dinamico ad aggiornamento asincrono.
- Garantire l'integrità, la consistenza e la sicurezza dei dati sfruttando in modo coordinato vincoli nativi del DBMS (chiavi, vincoli UNIQUE, CHECK, trigger) e controlli applicativi lato backend.

4. Utenti del sistema

Utente	Ruolo	Operazioni principali
Cliente / Passeggero	Utente pubblico registrato	Ricerca voli e verifica stato voli real-time (ritardi, partenze, ecc.), prenotazione, scelta posto, dichiarazione bagagli, pagamento, consultazione delle prenotazioni e aggiornamento dati personali.

Operatore voli	Dipendente aeroportuale	Visualizza e modifica i voli del proprio aeroporto (partenze e arrivi), in particolare gate, orari, ritardi e stato operativo.
Operatore bagagli	Dipendente aeroportuale	Gestisce i bagagli dichiarati dai passeggeri, conferma peso e tipologia al check-in, aggiorna lo stato del bagaglio.
Admin aeroporto	Responsabile operativo	Gestisce gli operatori del proprio aeroporto e accede alle funzionalità operative di voli e bagagli.
Amministratore database	Gestione iniziale del sistema	Inserisce dati strutturali come aeroporti, compagnie, aerei, gate e voli iniziali. Non è prevista un'interfaccia web dedicata a queste operazioni, in quanto riguardano la configurazione interna della piattaforma.

Gli operatori non possono registrarsi autonomamente: vengono creati da un amministratore. I passeggeri, invece, possono registrarsi autonomamente dal sito (come una classica piattaforma di prenotazioni-acquisto).

5. Analisi concettuale generale

Il sistema è stato suddiviso in aree funzionali, in modo da rappresentare correttamente i diversi aspetti del dominio aeroportuale.

Area funzionale	Entità coinvolte	Motivazione
Gestione utenti	Passeggero, Operatore, auth_user	Gestisce l'autenticazione e distingue tra clienti e personale aeroportuale.
Gestione aeroportuale	Aeroporto, Gate, Operatore	Associa gate e personale a uno specifico aeroporto. Ogni gate contiene terminal, area di imbarco e compatibilità con voli internazionali.
Gestione voli	Volo, Volo_Nazionale, Volo_Internazionale, Aereo, Compagnia_Aerea, Gate, Gestione_Volo	Rappresenta l'organizzazione operativa dei voli (distinti in nazionali e internazionali), degli aerei e delle modifiche effettuate dagli operatori.
Gestione prenotazioni	Prenotazione, Passeggero, Volo	Permette ai clienti di acquistare un posto su un volo.

Gestione bagagli	Bagaglio, Prenotazione, Operatore	Permette di dichiarare i bagagli in fase di prenotazione e validarli al check-in.
Gestione pagamenti	Transazione, Metodo_Pagamento, Prenotazione, Passeggero	Simula il pagamento della prenotazione e l'eventuale salvataggio di un metodo di pagamento.
Tabellone pubblico	Volo, Aeroporto, Gate	Mostra i voli giornalieri aggiornati dinamicamente.

6. Generalizzazione e Specializzazione

6.1 Generalizzazione Utente

Durante l'analisi è stata valutata l'introduzione di una gerarchia concettuale per l'entità Utente. Al fine di non duplicare le logiche di sicurezza e crittografia, si è scelto di delegare l'autenticazione al modulo nativo di Django (django.contrib.auth), il quale genera la tabella standard auth_user.

Le entità applicative Passeggero (dati anagrafici, passaporto, codice fiscale) e Operatore (ruolo operativo, scalo di appartenenza) sono mantenute distinte tra loro e collegate a auth_user mediante una relazione uno-a-uno (1:1) tramite la chiave esterna id_user FK UNIQUE. Questo isola i dati personali dei clienti dai profili lavorativi dei dipendenti, sfruttando il collaudato sistema di sessioni e hashing del framework.

6.2 Specializzazione Volo

La specializzazione dell'entità Volo nelle sotto-entità Volo_Nazionale e Volo_Internazionale risponde alla necessità di evitare la proliferazione di valori nulli (*NULL*) all'interno della tabella principale.

Dal punto di vista concettuale, la gerarchia è **totale e disgiunta**:

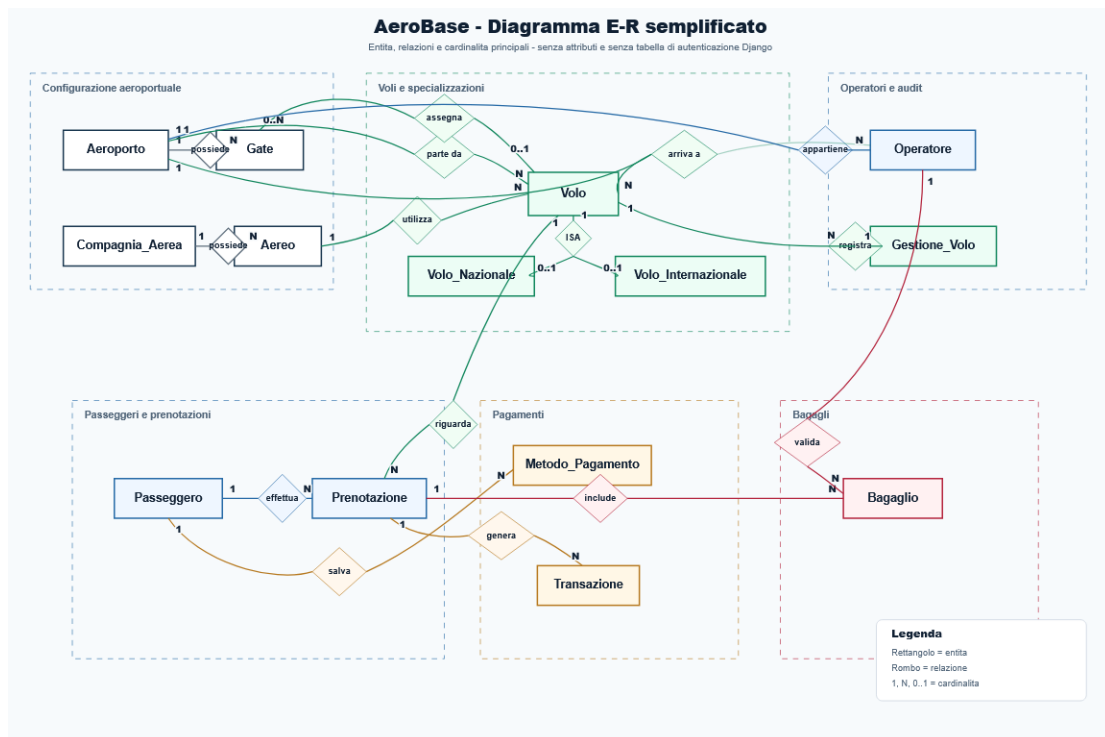
- **Totale:** Ogni volo programmato deve appartenere tassativamente a una delle due sottoclassi (non esistono voli "neutri").
- **Disgiunta:** Un volo non può essere simultaneamente nazionale e internazionale.

A livello di modello logico, la specializzazione è tradotta nel pattern Table-per-Subclass: è presente una tabella generale (VOLO) che ospita gli attributi condivisi (orari, aeroporti di origine e destinazione, aereo, gate, prezzo, stato e l'attributo discriminante tipo_volo), e due tabelle specifiche (VOLO_NAZIONALE, VOLO_INTERNAZIONALE). In queste ultime, il campo id_volo assume il doppio ruolo di Chiave Primaria (PK) e Chiave Esterna (FK) puntando a VOLO.

Mentre la disgiunzione viene blindata a livello di database mediante l'uso di Trigger dedicati, la totalità viene implementata a livello applicativo racchiudendo l'inserimento del volo generico e del suo dettaglio specializzato all'interno della medesima transazione atomica ACID.

7. Diagramma E-R

Il diagramma E-R rappresenta le principali entità del sistema e le loro relazioni, descrive visivamente come sono organizzate le informazioni all'interno del database e in che modo i diversi elementi comunicano tra loro.



Una versione grafica del diagramma è stata realizzata tramite draw.io (presente nel repository del progetto).

8. Entità e attributi

8.1 Tabelle Classiche

Aeroporto

Attributo	Tipo	Descrizione
codice_iata	PK	Codice IATA dell'aeroporto
nome_aeroporto	VARCHAR	Nome completo dell'aeroporto
citta	VARCHAR	Città dell'aeroporto
nazione	VARCHAR	Nazione
codice_icao	UNIQUE	Codice ICAO, opzionale

Compagnia_Aerea

Attributo	Tipo	Descrizione
codice_vettore	PK	Codice identificativo della compagnia
nome_compagnia	VARCHAR	Nome della compagnia
nazione	VARCHAR	Nazione della compagnia

Aereo

Attributo	Tipo	Descrizione
-----------	------	-------------

Gate

Attributo	Tipo	Descrizione
-----------	------	-------------

id_aereo	PK	Identificativo dell'aereo
modello	VARCHAR	Modello dell'aereo
capacita_passeggeri	INT	Numero massimo di passeggeri
codice_vettore	FK	Compagnia aerea associata

codice_gate	PK	Codice del gate
codice_aeroporto	FK	Aeroporto a cui appartiene il gate
terminal	VARCHAR	Terminal del gate

Operatore

Attributo	Tipo	Descrizione
codice_operatore	PK	Identificativo operatore
nome	VARCHAR	Nome
cognome	VARCHAR	Cognome
email	UNIQUE	Email
cellulare	UNIQUE	Numero cellulare
ruolo	ENUM	Admin, operatore voli, operatore bagagli
codice_aeroporto	FK	Aeroporto di appartenenza
id_user	FK UNIQUE	Collegamento a auth_user

Passeggero

Attributo	Tipo	Descrizione
username	PK	Identificativo del passeggero
nome	VARCHAR	Nome
cognome	VARCHAR	Cognome
email	UNIQUE	Email
numero_passaporto	UNIQUE, NULL	Numero passaporto
cellulare	UNIQUE, NULL	Numero di cellulare
nazionalita	VARCHAR	Nazionalità
id_user	FK UNIQUE	Collegamento a auth_user

Bagaglio

Attributo	Tipo	Descrizione
id_bagaglio	PK	Identificativo del bagaglio
peso_kg	DECIMAL, NULL	Peso verificato al check-in
tipo	ENUM	Cabina, stiva, speciale
username_passeggero	FK	Passeggero associato
id_volo	FK	Volo associato
codice_operatore	FK, NULL	Operatore che verifica
stato	ENUM	Prenotato, imbarcato, consegnato, smarrito, ritrovato

Metodo_Pagamento

Attributo	Tipo	Descrizione
id_metodo	PK	Identificativo metodo
username_passeggero	FK	Passeggero proprietario
intestatario	VARCHAR	Intestatario carta
ultime_cifre	CHAR	Ultime 4 cifre
mese_scadenza	INT	Mese scadenza
anno_scadenza	INT	Anno scadenza
token_pagamento	VARCHAR	Token simulato
data_registrazione	TIMESTAMP	Data salvataggio

data_aggiornamento	TIMESTAM P	Ultimo aggiornament o
---------------------------	---------------	-----------------------------

Transazione

Attributo	Tipo	Descrizione
id_transazione	PK	Identificativo transazione
username_passeggero	FK	Passeggero prenotazione
id_volo	FK	Volo prenotazione
importo	DECIMAL	Importo pagato
id_transazione_esterno	UNIQUE	Codice simulato
metodo_usato	VARCHAR	Metodo pagamento
stato	ENUM	In attesa, completato, fallito
data_pagamento	TIMESTAM P	Data pagamento

Volo

Attributo	Tipo	Descrizione
id_volo	PK	Identificativo volo
numero_volo	UNIQUE	Numero commerciale
orario_partenza	DATETIME	Data e ora partenza
orario_arrivo	DATETIME	Data e ora arrivo
partenza	FK	Aeroporto partenza
destinazione	FK	Aeroporto arrivo
id_aereo	FK	Aereo utilizzato
codice_gate	FK	Gate assegnato
stato	ENUM	Stato volo
ritardo_minuti	INT	Ritardo minuti
prezzo	DECIMAL	Prezzo biglietto

8.2 Tabelle di Specializzazione Volo

Volo_Nazionale			Volo_Internazionale		
Attributo	Tipo	Descrizione	Attributo	Tipo	Descrizione
id_volo	PK, FK	Riferimento uno-a-uno al volo generale	id_volo	PK, FK	Riferimento uno-a-uno al volo generale
agevolazioni_statali	BOOLEAN	Indica se il volo prevede agevolazioni statali	richiede_passaporto	BOOLEAN	Indica se è richiesto il passaporto
tipo_agevolazione	VARCHAR, NULL	Descrizione dell'eventuale agevolazione	tipo_visto	VARCHAR, NULL	Eventuale visto richiesto
			validita_minima_passaporto_mesi	INT	Validità minima del passaporto
			fuso_orario_destinazione	VARCHAR, NULL	Fuso orario della destinazione
			certificazioni_sanitarie_richieste	TEXT, NULL	Eventuali certificazioni sanitarie richieste

8.3 Relazioni con attributi

Prenotazione			Gestione_Volo		
Attributo	Tipo	Descrizione	Attributo	Tipo	Descrizione
id_prenotazione	UNIQUE	Identificativo applicativo della prenotazione	id_gestione	PK	Identificativo del log
username_passeggero	PK, FK	Passeggero che prenota	codice_operatore	FK	Operatore che modifica il volo
id_volo	PK, FK	Volo prenotato	id_volo	FK	Volo modificato
data_acquisto	TIMESTAMP	Data della prenotazione	timestamp_modifica	DATE TIME	Data e ora della modifica
posto	CHAR	Posto assegnato	tipo_operazione	ENUM	Tipo di modifica effettuata
classe	ENUM	Economy, business, first			
stato_pagamento	ENUM	Non pagato, pagato, rimborsato			

username_passeggero: La chiave primaria logica della prenotazione è la coppia (*username_passeggero*, *id_volo*), perché un passeggero non può prenotare due volte lo stesso volo. Il campo *id_prenotazione* è stato mantenuto come identificativo aggiuntivo utile lato applicativo.

9. Progettazione logica relazionale

Dal modello concettuale si ricavano le seguenti relazioni:

AEROPORTO (*codice_iata* PK, *nome_aeroporto*, *citta*, *nazione*, *codice_icao* UNIQUE)

COMPAGNIA_AEREA (*codice_vettore* PK, *nome_compagnia*, *nazione*)

AEREO (*id_aereo* PK, *modello*, *capacita_passeggeri*, *codice_vettore* FK)

GATE (*codice_gate PK, codice_aeroporto FK, terminal*)

VOLO (*id_volo PK, numero_volo UNIQUE, orario_partenza, orario_arrivo, partenza FK, destinazione FK, id_aereo FK, codice_gate FK, stato, ritardo_minuti, prezzo*)

VOLO_NAZIONALE(*id_volo PK/FK, agevolazioni_statali, tipo_agevolazione*)

VOLO_INTERNAZIONALE(*id_volo PK/FK, richiede_passaporto, tipo_visto, validita_minima_passaporto_mesi, fuso_orario_destinazione, certificazioni_sanitarie_richieste*)

PASSEGGERO (*username PK, nome, cognome, email UNIQUE, numero_passaporto UNIQUE, cellulare UNIQUE, nazionalita, id_user FK UNIQUE*)

OPERATORE (*codice_operatore PK, nome, cognome, email UNIQUE, cellulare UNIQUE, ruolo, codice_aeroporto FK, id_user FK UNIQUE*)

PRENOTAZIONE (*username_passeggero PK/FK, id_volo PK/FK, id_prenotazione UNIQUE, data_acquisto, posto, classe, stato_pagamento*)

BAGAGLIO (*id_bagaglio PK, peso_kg, tipo, username_passeggero FK, id_volo FK, codice_operatore FK, stato, data_aggiornamento*)

GESTIONE_VOLO (*id_gestione PK, codice_operatore FK, id_volo FK, timestamp_modifica, tipo_operazione*)

METODO_PAGAMENTO (*id_metodo PK, username_passeggero FK, intestatario, ultime_cifre, mese_scadenza, anno_scadenza, token_pagamento, data_registrazione*)

TRANSAZIONE (*id_transazione PK, username_passeggero FK, id_volo FK, importo, id_transazione_esterno UNIQUE, metodo_usato, stato, data_pagamento*)

Nota: la tabella auth_user è generata da Django e viene utilizzata per autenticazione, password hashate e sessioni.

10. Vincoli del sistema ed implementazione nel DB MySQL

10.1 Vincoli di Integrità e Unicità

L'integrità referenziale è assicurata da vincoli di chiave esterna (FOREIGN KEY) che impediscono orfani e disallineamenti logici (es. un'istanza di Bagaglio punta rigidamente alla chiave composta (username_passeggero, id_volo) di una Prenotazione esistente).

I vincoli UNIQUE proteggono i dati sensibili da inserimenti duplicati, bloccando sovrapposizioni su e-mail, numeri di passaporto, codici fiscali, numeri commerciali dei voli (numero_volo) e token transazionali esterni.

10.2 Vincoli di Dominio e Vincoli CHECK

I campi a spettro valoriale predefinito sono forzati mediante domini ENUM (es. i ruoli dell'operatore, le classi di prenotazione, gli stati del volo e del bagaglio). L'integrità semantica dei dati numerici e temporali viene invece validata a livello di engine di database attraverso appositi vincoli CHECK:

- Capacità passeggeri di un aereo > 0
- Orario di arrivo $>$ Orario di partenza
- Ritardo in minuti ≥ 0
- Aeroporto di Partenza $\neq$ Aeroporto di Destinazione
- $1 \leq$ Mese di scadenza pagamento ≤ 12

10.3 Vincoli sulle Specializzazioni e Trigger SQL

Per blindare la disgiunzione e la coerenza della specializzazione dell'entità Volo direttamente a livello fisico, sono stati implementati dei trigger BEFORE INSERT sulle tabelle figlie. Questi bloccano l'operazione lanciando un segnale d'errore (SIGNAL SQLSTATE '45000') qualora si tenti di violare le regole di business.

```

DELIMITER //

-- Trigger per l'inserimento nella tabella Volo_Nazionale
CREATE TRIGGER trg_vn_before_insert
BEFORE INSERT ON Volo_Nazionale
FOR EACH ROW
BEGIN
    -- 1. Controllo di disgiunzione: verifica che il volo non esista in Volo_Internazionale
    IF EXISTS (
        SELECT 1 FROM Volo_Internazionale
        WHERE id_volo = NEW.id_volo
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Errore di integrità: Il volo selezionato è già censito come internazionale.';
    END IF;

    -- 2. Controllo di coerenza: verifica che il tipo_volo nella tabella madre sia 'nazionale'
    IF NOT EXISTS (
        SELECT 1 FROM Volo
        WHERE id_volo = NEW.id_volo
        AND tipo_volo = 'nazionale'
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Errore di coerenza: Il campo tipo_volo nel record madre deve essere
"nazionale".';
    END IF;
END//

-- Trigger per l'inserimento nella tabella Volo_Internazionale
CREATE TRIGGER trg_vi_before_insert
BEFORE INSERT ON Volo_Internazionale
FOR EACH ROW
BEGIN
    -- 1. Controllo di disgiunzione: verifica che il volo non esista in Volo_Nazionale
    IF EXISTS (
        SELECT 1 FROM Volo_Nazionale
        WHERE id_volo = NEW.id_volo
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Errore di integrità: Il volo selezionato è già censito come nazionale.';
    END IF;

    -- 2. Controllo di coerenza: verifica che il tipo_volo nella tabella madre sia 'internazionale'
    IF NOT EXISTS (
        SELECT 1 FROM Volo
        WHERE id_volo = NEW.id_volo
        AND tipo_volo = 'internazionale'
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Errore di coerenza: Il campo tipo_volo nel record madre deve essere
"internazionale".';
    END IF;
END//
DELIMITER ;

```

Il flusso applicativo inserisce i dati secondo uno schema transazionale che garantisce l'atomicità e preserva il vincolo di totalità:

```

START TRANSACTION;

-- Inserimento dell'entità generale in Volo
INSERT INTO Volo (numero_volo, orario_partenza, orario_arrivo, partenza, destinazione, id_aereo,
codice_gate, stato, ritardo_minuti, prezzo, tipo_volo)
VALUES ('AZA777', '2026-07-10 10:00:00', '2026-07-10 11:10:00', 'FCO', 'LIN', 1, 'FCO-A10', 'in_orario',
0, 95.00, 'nazionale');

-- Recupero dell'ID autogenerato
SET @id_volo = LAST_INSERT_ID();

-- Inserimento contestuale nella specifica tabella di specializzazione
INSERT INTO Volo_Nazionale (id_volo, agevolazioni_statali, tipo_agevolazione)
VALUES (@id_volo, 0, NULL);

COMMIT;

```

10.4 Politiche di Cancellazione (ON DELETE)

Le direttive di cancellazione cambiano a seconda della dipendenza logica:

- **ON DELETE CASCADE:** Applicata alle tabelle di specializzazione (Volo_Nazionale, Volo_Internazionale), alle prenotazioni e ai dettagli transazionali. Se l'entità principale viene rimossa, i record dipendenti vengono eliminati automaticamente.
- **ON DELETE SET NULL:** Utilizzata per mantenere intatto lo storico operativo. Se un profilo Operatore viene rimosso, il riferimento sul Bagaglio da lui controllato si azzerà, preservando il record logistico.
- **ON DELETE RESTRICT:** Impedisce la cancellazione di entità strutturali (es. Aeroporto o Gate) se vi sono voli o operatori ancora collegati.

10.5 Indici e Ottimizzazione

Per minimizzare i tempi di esecuzione delle query più frequenti, sono stati definiti indici di ricerca dedicati sui campi selettivi:

- `idx_volo_orario_partenza` e `idx_volo_stato` (ottimizzano la ricerca voli e il tabellone pubblico).
- `idx_bagaglio_stato` (accelera i filtri nella dashboard check-in).
- `idx_prenotazione_data_acquisto` (velocizza i report commerciali).
- `idx_gate_aeroporto` e `idx_gestione_operatore_volo` (migliorano l'isolamento multi-tenant e l'estrazione degli audit log).

11. Vincoli applicativi gestiti da Django

Alcune regole di business flessibili o dipendenti dal contesto dinamico sono demandate al backend dell'applicazione web:

- **Gestione dei posti:** L'assegnazione della stringa del posto viene memorizzata in Prenotazione, ma la validazione dinamica (verifica della mappa dei posti dell'aereo, controllo delle sovrapposizioni e corrispondenza con la classe scelta) è governata da metodi Django.
- **Flusso dei pagamenti:** La sottomissione del pagamento coordina in modo sicuro la creazione del record Transazione e la mutazione dello stato della prenotazione in pagato.
- **Workflow dei bagagli:** Django gestisce lo stato del ciclo di vita del bagaglio. Al momento della prenotazione il passeggero lo dichiara (stato prenotato, peso non inserito); al banco di accettazione l'operatore immette il peso reale e il backend registra la firma del dipendente mutando lo stato in imbarcato.
- **Filtri di compatibilità dei gate:** La corrispondenza geografica e logica (es. impedire l'assegnazione di un gate nazionale a un volo transoceanico internazionale) è presa in carico dai form di validazione Django.

12. Gestione ruoli e isolamento multi-tenant

L'architettura multi-tenant è strutturata iniettando la chiave esterna codice_aeroporto all'interno dell'entità Operatore. Al momento del login, il middleware o la vista di Django intercettano lo scalo di appartenenza dell'utente dello staff. Ogni successiva interrogazione al database viene filtrata nativamente (`.filter(partenza=operatore.codice_aeroporto)`):

- Un operatore dello scalo di Fiumicino (FCO) può visualizzare, tracciare e modificare esclusivamente i voli e i bagagli inerenti a FCO.
- Un amministratore locale di un aeroporto può creare o modificare account afferenti unicamente al proprio codice IATA.

L'isolamento dei dati viene così garantito a livello applicativo, evitando che utenti di un determinato scalo possano accedere alle informazioni riservate di un'altra infrastruttura concorrente.

13. Implementazione del sistema informativo

La soluzione è strutturata secondo il paradigma MVT (Model-View-Template) tipico di Django, integrato con database MySQL esterno.

La separazione dei componenti prevede:

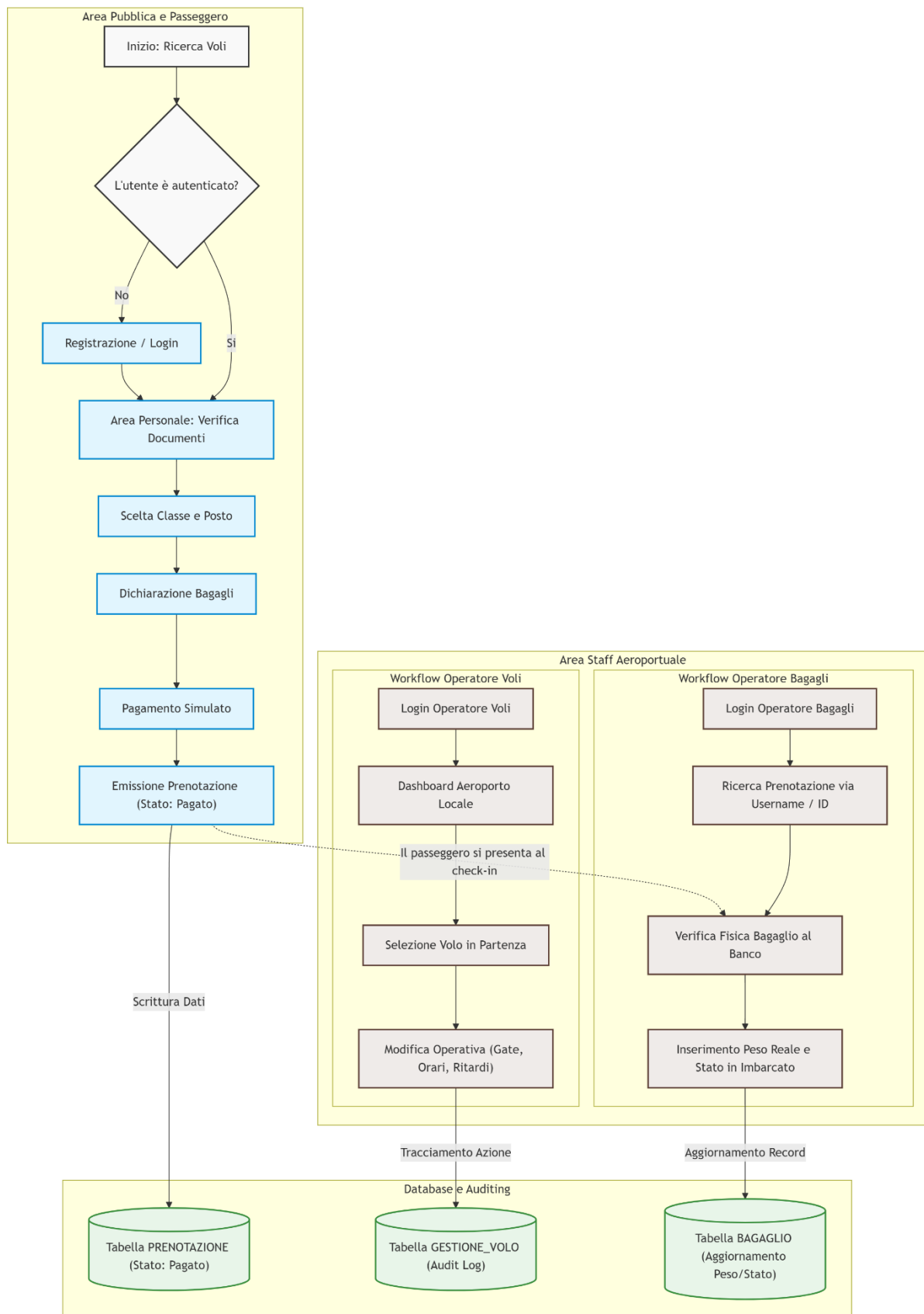
- **Modelli (models.py):** Definiscono le strutture dati speculari alle tabelle MySQL descritte nello schema logico, includendo le definizioni dei vincoli e delle relazioni OneToOneField, ForeignKey e ManyToManyField.
- **Form (forms.py):** Gestiscono l'input-validation sia per i passeggeri (documenti, prenotazione) sia per gli operatori (inserimento ritardi, modifica voli).

- **Viste (views.py):** Sviluppano la logica di business, orchestrano le transazioni sul database e applicano i decorator di sicurezza (es. @login_required e controlli personalizzati sui gruppi/ruoli).
- **Template HTML e statici (templates, static):** Compongono l'interfaccia utente responsive, supportata da script **JavaScript** asincroni dedicati alla comunicazione via API verso il backend per il refresh asincrono del tabellone voli.

Tabella riassuntiva componenti e scopo

Componente	Ruolo
Python	Linguaggio principale del backend
Django	Framework web, gestione URL, view, form, autenticazione e template
MySQL	Database relazionale del progetto
HTML	Struttura delle pagine
CSS	Grafica e layout dark mode
JavaScript	Aggiornamento dinamico del tabellone voli
GitHub	Versionamento e consegna del progetto
draw.io	Realizzazione del diagramma E-R

14. Workflow del sistema



1. **Workflow Cliente:** Il passeggero effettua la ricerca. Se decide di procedere all'acquisto, esegue il login o si registra. Il sistema verifica la presenza dei documenti idonei nel profilo (es. passaporto valido per tratte internazionali), fa selezionare posto e bagagli, richiede i dati della carta e chiude la transazione emettendo la prenotazione.
2. **Workflow Operatore Voli:** Il dipendente accede alla dashboard filtrata per il proprio scalo. Può selezionare un volo in partenza e aggiornare i minuti di ritardo o mutare lo stato (es. da in orario a imbarco). Ogni operazione genera un record automatico nella tabella di audit Gestione_Volo.
3. **Workflow Operatore Bagagli:** L'operatore interroga il database inserendo l'ID prenotazione o lo username del passeggero presentatosi al banco del check-in. Prende in carico il bagaglio contrassegnato come prenotato, ne pesa l'entità fisica sul rullo, aggiorna il valore dei chilogrammi e ne muta lo stato in imbarcato.

15. Motivazioni progettuali e architetturali

L'architettura del sistema AeroBase è guidata dal principio della **separazione delle responsabilità** (*Separation of Concerns*).

Le fondamenta strutturali, la persistenza e la coerenza relazionale dei dati sono blindate all'interno del DBMS MySQL tramite l'uso rigoroso di chiavi referenziali, vincoli CHECK, indici e trigger sulle specializzazioni. Questa scelta assicura che lo stato del database rimanga integro e consistente a prescindere da qualunque agente o script esterno che tenti di interrogarlo o modificarlo.

Al contrario, i flussi di lavoro flessibili, le dinamiche di interfaccia, l'autenticazione crittografica e le restrizioni di isolamento multi-tenant sono affidate al backend Django. L'adozione di `auth_user` evita la riscrittura di algoritmi di hashing per le password, innalzando gli standard di sicurezza. L'adozione del pattern *Table-per-Subclass* per l'entità Volo garantisce un database pulito, normalizzato e facilmente estensibile.

16. Conclusione

AeroBase si presenta come una soluzione ERP solida, scalabile e bilanciata per la gestione di un contesto aeroportuale multi-tenant semplificato. Il progetto dimostra come una corretta analisi concettuale e una metodica progettazione logica possano tradursi in una base dati relazionale efficiente e coerente, capace di cooperare con un framework moderno.

L'introduzione della specializzazione dei voli, l'adozione dei trigger SQL per l'applicazione rigorosa dei vincoli di disgiunzione e la gestione coordinata dei workflow applicativi assicurano la stabilità del software. La struttura modulare del codice e la pulizia del modello relazionale consentono ad AeroBase di porsi come una base ottimale per future espansioni, quali l'integrazione di gateway di pagamento reali, sistemi di notifica automatica via e-mail e algoritmi per l'assegnazione automatizzata dei gate d'imbarco.